

Short-Term Electrical Load Forecasting for a 220kV Substation: A Comparative Study of Classical, Machine Learning, and Attention-Based Seq2Seq LSTM Models

Ayush Muttepawar
EE Department, IIT Kanpur
ayushmg24@iitk.ac.in

Shubhang Mani Tripathi
EE Department, IIT Kanpur
shubhangm24@iitk.ac.in

Orugonda Chakresh
EE Department, IIT Kanpur
chakresh24@iitk.ac.in

Abstract. Accurate electricity price and demand forecasting is essential for grid operators, energy traders, and decision makers, but it is challenging to achieve. In markets like the Indian Energy Exchange, prices can change quickly due to demand changes, renewable energy variability, fuel costs, and bidding strategies, making day-ahead forecasting complex. In this study, we compare five different modeling approaches to predict hourly load at the 220kV Kuppam substation using data from December 2023 to November 2025. We tested a classical statistical baseline (SARIMA), two standard machine learning models (XGBoost and Support Vector Regression), a Stacked LSTM using recursive inference, and a Sequence-to-Sequence (Seq2Seq) LSTM with Luong attention. Each model was evaluated under two setups: a univariate configuration using only past load data, and a multivariate configuration that added weather variables and calendar patterns. Ultimately, our findings suggest that while tree-based models like XGBoost are fast and effective for single-step predictions, attention-based sequence models are much better at handling multi-step forecasts for real-world grid operations.

I. INTRODUCTION

Indian distribution utilities need accurate substation-level load forecasts to schedule generation, manage procurement, and keep the grid stable. The hourly load series has strong daily and weekly cycles, reacts to weather, and often spikes nonlinearly during peak hours. Our goal here was to build a forecasting system that can actually be deployed and that handles these conditions without falling apart during the sharper swings.

A. Motivation

Better forecasts pay off directly: utilities can reduce peak-load stress, plan maintenance windows, schedule generation more efficiently, and keep the grid more reliable overall. Substation load is noisy and its behavior shifts with weather and the calendar (weekdays, weekends, holidays), which is exactly the kind of nonlinearity that linear statistical models have difficulty with.

We compare several modeling paradigms, from classical autoregressive models to gradient-boosted ensembles to recurrent neural networks, to see which techniques and feature representations actually work for this problem.

We also spent time earlier on short-term Market Clearing Price (MCP) forecasting using Indian Energy Exchange (IEX) data. Price and demand share a similar hourly structure driven by weather and the calendar, so we used the same classical decomposition and ARMA/SARIMAX approach there first. That work made it clear that linear models miss the non-

linear volatility in power-sector time series, and that finding shaped how we approached the substation problem. The rest of this report is entirely about substation demand forecasting.

B. Approach overview

We study hourly load forecasting at the 220kV Kuppam substation, starting from a classical statistical baseline and working up to deep learning structures. Five modeling paradigms are evaluated:

1. Seasonal Autoregressive Integrated Moving Average (SARIMA)
2. Gradient Boosted Decision Trees (XGBoost Regressor)
3. Support Vector Regression (SVR)
4. Stacked LSTM with rain-aware recursive inference
5. Seq2Seq LSTM encoder-decoder with and without Luong attention

We test each model under two feature configurations: univariate (load history only) and multivariate (weather and calendar added). The exogenous weather variables, temperature, relative humidity, and precipitation, come from the Open-Meteo API.

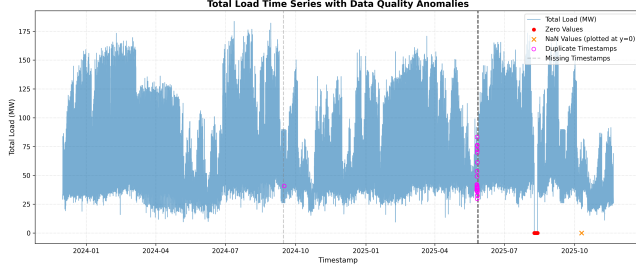


Figure 1: Raw total load time series (December 2023 to November 2025) with data quality anomalies flagged: duplicate timestamps (magenta circles), a stretch of zero-load readings (red), an isolated NaN value (orange), and a run of missing hourly timestamps (dashed vertical line) around May to June 2025.

II. DATASET AND PREPROCESSING

A. Data source and cleaning

The dataset contains hourly total load (MW) at the 220kV Kuppam substation from December 1, 2023 to November 20, 2025. Exploratory analysis turned up a few data quality issues: 25 duplicated timestamps (50 rows total), 103 zero-load instances, and two missing load values.

We reindexed all records to a strict hourly frequency (`asfreq('1H')`) to flag missing hours, then imputed missing values with linear interpolation combined with forward and backward filling. That gave us with a continuous dataset of 17,304 hourly records for training.

Fig. 1 shows the raw series before cleaning. The duplicate timestamps cluster almost entirely in a narrow band in mid-2025, and the zero-load run flagged in August 2025 is long enough that it probably is not genuine zero demand. Both look more like sensor or telemetry outages than real operating states, which is why we treated them as missing and imputed them rather than keeping the observed zeros.

B. Exogenous weather data

We pulled historical hourly weather data for the Kuppam substation (lat 12.75, lon 78.34) from the Open-Meteo API: surface temperature, relative humidity, cloud cover, and a binary precipitation indicator. This was merged into the load series with a left join on timestamp, and any gaps were forward-filled.

C. Train, validation, and test splits

We split the data chronologically to keep the time order intact. Training covers everything before April 2025, validation runs from April to August 2025, and the test set covers August 2025 onward. We also held out a blind one-week horizon specifically to evaluate the Stacked LSTM.

III. FEATURE ENGINEERING

The same set of engineered features was used across models, with minor adaptations for each architecture:

- Autoregressive lags: `lag_1`, `lag_12`, `lag_24`, `lag_48`, and `lag_168`, covering hourly, half-daily, daily, bi-daily, and weekly dependencies, plus the 24-hour rolling mean and standard deviation.
- Calendar features: normalized hour-of-day and day-of-week (with cyclical sine and cosine encodings), month, and a binary flag for Indian public holidays.
- Weather features: temperature, relative humidity, cloud cover, and a rain flag from Open-Meteo.

`hour_cos` (correlation -0.618 , mutual information 0.348) and relative humidity (correlation -0.463) turned out to be the two strongest individual predictors of load, so both were kept across all model families.

Table 1: Linear Correlation and Mutual Information of Exogenous and Cyclical Features with Total Substation Load

Feature Variable	Correlation Coefficient	Mutual Information
<code>hour_cos</code>	-0.6183	0.3482
<code>relative_humidity_2m</code>	-0.4634	0.1713
<code>temperature_2m</code>	0.3456	0.1681
<code>hour_sin</code>	0.0653	0.1438
<code>dow_sin</code>	0.0093	0.0078
<code>dow_cos</code>	0.0012	0.0011

IV. CLASSICAL STATISTICAL BASELINE: SARIMA

A. Preliminary validation study on a public dataset

Before committing to a seasonal specification for the substation series, we ran a preliminary week-one study on a public univariate electricity time series (the Kaggle Electric Production dataset, monthly observations from 1985 to 2018) to confirm that explicit seasonal modeling was warranted and to establish a classical-model workflow. The series contained 397 observations, split chronologically into 297 training and 100 testing points, with no missing values.

Visual inspection showed a clear upward trend along with a recurring annual cycle, motivating a comparison between a non-seasonal ARIMA model and a seasonal ARIMA (SARIMA) model. A multiplicative decomposition ($Y_t = T_t \cdot S_t \cdot R_t$) showed a smooth long-term trend, a strongly periodic seasonal component, and a residual component with comparatively low variance, indicating that most of the series' variation came from seasonality rather than noise.

4.1.1 Stationarity analysis

We assessed stationarity with the Augmented Dickey-Fuller (ADF) test. On the original series, the ADF statistic was 2.0124 with a p-value of 0.2812, so the null hypothesis of a unit root could not be rejected and the series was treated as non-stationary. After first-order differencing ($Y_t = Y_t - Y_{t-1}$), the ADF statistic fell to 6.0914 with a p-value of 1.03×10^{-7} , confirming stationarity and fixing $d = 1$. A subsequent

seasonal-differencing check at lag 12 gave an ADF statistic of 5.0184 ($p = 2.04 \times 10^5$), and the resulting ACF/PACF decayed faster than in the non-seasonally-differenced case, confirming seasonal structure and supporting a seasonal differencing specification.

4.1.2 Model selection

We compared four candidate ARIMA(p, d, q) specifications by AIC and BIC (Table 2). ARIMA(2,1,2) had the lowest values on both criteria and was selected as the final non-seasonal model.

Table 2: ARIMA Candidate Model Comparison (Preliminary Study)

Model	AIC	BIC
ARIMA(1,1,1)	1900.98	1912.05
ARIMA(2,1,1)	1639.30	1654.06
ARIMA(1,1,2)	1811.95	1826.71
ARIMA(2,1,2)	1525.72	1544.18

Residual diagnostics on the fitted ARIMA(2,1,2) model showed errors centered near zero with no obvious trend. The Ljung-Box test gave $p = 0.338$ at lag 10 and $p = 0.118$ at lag 20. Since both exceed 0.05, the null hypothesis of no residual autocorrelation could not be rejected, so the residuals were approximately white noise and the model had captured most of the temporal structure in the training data.

4.1.3 ARIMA vs. SARIMA performance

On the 100-point test set, the ARIMA(2,1,2) model gave an MAE of 5.21, an RMSE of 6.51, a MAPE of 4.91%, and an R^2 of 0.5673. A SARIMA model built on the same seasonal structure improved substantially on this baseline, as shown in Table 3. Table 3.

Table 3: Standard ARIMA vs. Seasonal ARIMA (Preliminary Study)

Metric	ARIMA(2,1,2)	SARIMA
MAE	5.2144	2.7248
RMSE	6.5062	3.5255
MAPE	4.91%	2.63%
R^2	0.5673	0.8730

Adding seasonality nearly halved both the MAE and RMSE and raised the explained variance from 56.7% to 87.3%. Seasonality accounted for most of the predictable structure in this series, so modeling it explicitly, rather than relying on a non-seasonal ARIMA fit, was necessary to reach acceptable accuracy. This is why we used a seasonal model with an explicit 24-hour periodicity on the substation data below.

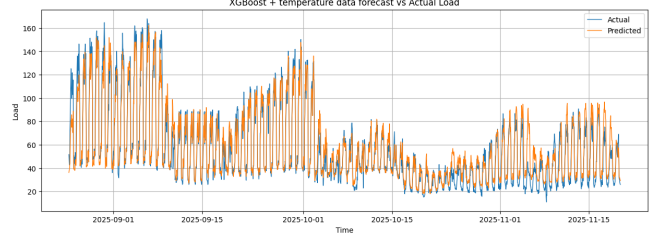


Figure 2: Weather-enabled XGBoost (Version 3) forecast versus actual substation load over the held-out test period (August to November 2025). The model tracks the diurnal cycle closely, with the largest deviations occurring around sharp demand jumps.

B. Application to substation load

Building on this result, we fit a Seasonal Autoregressive Integrated Moving Average (SARIMA) model to the substation load series as a linear baseline. We applied first-order differencing ($d = 1$) to remove the trend and seasonal differencing ($D = 1, s = 24$) to remove the 24-hour cycle. The autocorrelation function of the differenced series showed a spike at lag 24, validating our seasonal parameters. SARIMA captures the trend and daily seasonality but misses the nonlinear, weather-driven spikes in the load series. This limitation is consistent with the moderate ARIMA test performance seen in the preliminary study, and it is why we moved on to evaluate machine learning and deep learning approaches.

V. MACHINE LEARNING BASELINES

A. XGBoost regressor

We used an XGBoost regressor to model nonlinear associations between the autoregressive lag features and the weather variables.

The baseline configuration (Version 3) used 3,000 estimators, a learning rate of 0.03, and a maximum tree depth of 6. The weather-enabled variant increased the estimators to 10,000, lowered the learning rate to 0.005, and raised the depth to 8.

For Version 3, the baseline model without weather data had a Mean Absolute Error (MAE) of 8.32 MW and a test R^2 of 0.8513. Adding weather data gave a mixed result: MAE rose slightly to 8.53 MW while test R^2 improved marginally to 0.8517. The two metrics moved in opposite directions, which suggests the weather features had little practical effect on this model at the depths and learning rates we tested.

B. Support vector regression

We also tested an ϵ -SVR model with a Radial Basis Function (RBF) kernel: error penalty (C) of 50, kernel smoothness (γ) of 0.01, and error margin (ϵ) of 0.01. Input variables were standardized with `StandardScaler`, fit only on the training split.

Without weather features, SVR recorded an MAE of 9.44 MW and a test R^2 of 0.8341. With weather features added,

SVR Pipeline

Load and clean time series (deduplicate, enforce hourly frequency)
→ merge exogenous weather (left join, fill) → construct features
(lags, cyclical time encodings) → chronological split (train < Apr'25, val Apr to Aug'25, test ≥ Aug'25) → build X (feature matrix), y (total load) → scale X, y (StandardScaler fit on train only)
→ fit RBF-kernel SVR ($C = 50, \gamma = 0.01, \epsilon = 0.01$) → predict on test set (inverse-transform) → evaluate (MAE, RMSE, R^2)

Figure 3: SVR pipeline.

Stacked LSTM Pipeline

Phase 1: Ingest hourly load and temperature, forward-filling any sensor gaps.
Phase 2: Engineer cyclical time encodings alongside weekend and holiday flags.
Phase 3: Scale inputs to $[0,1]$ using MinMaxScaler. The input vector includes load, temperature, precipitation, normalized hour-of-day, normalized day-of-week, and a binary holiday indicator.
Phase 4: Pass inputs through an LSTM(64) layer, a 20% dropout layer, an LSTM(32) layer, and a dense output layer.
Phase 5: Predict step $t + 1$, append the prediction to the history, and repeat 168 times for the full week.
Phase 6: Inverse-scale the results to MW and evaluate using MAE, MAPE, R^2 , and residual diagnostics.

Figure 4: Stacked LSTM load forecasting pipeline.

MAE rose to 10.24 MW and test R^2 fell to 0.7801.

So SVR actually did better without the weather features. The algorithm is sensitive to feature scaling and dimensionality, and adding raw weather variables without any specific weighting just increased the input space and added noise. The kernel ended up fitting false correlations rather than the real relationship between load and weather.

C. Stacked LSTM using recursive inference

We built a stacked recurrent network with two LSTM layers: the first with 64 units, returning sequences into a second layer with 32 units, with a 20% dropout layer in between for regularization. Training used Adam with a 0.0005 learning rate.

Model inputs were scaled load, temperature, precipitation, normalized hour-of-day, normalized day-of-week, and a binary holiday indicator.

We tested the model using recursive inference over a blind 168-hour horizon. At each step the model predicts load at $t + 1$, that prediction is inverse-scaled, appended to the historical record, and used to rebuild the autoregressive lag features for the next step. Combined with the known future weather and calendar variables, this lets the model predict $t + 2$, and the loop repeats across the full week.

The stacked LSTM reached an MAE of 6.53 MW and a test R^2 of 0.8172 over the blind horizon. Residual diagnostics showed some structure still left in the errors: the Ljung-Box test returned a p -value of 0.0000, pointing to residual autocorrelation, and the Shapiro-Wilk test returned a p -value of 0.0005, meaning the error distribution was not normal.

Attention Seq2Seq Grid Load Forecasting Diagnostic Dashboard

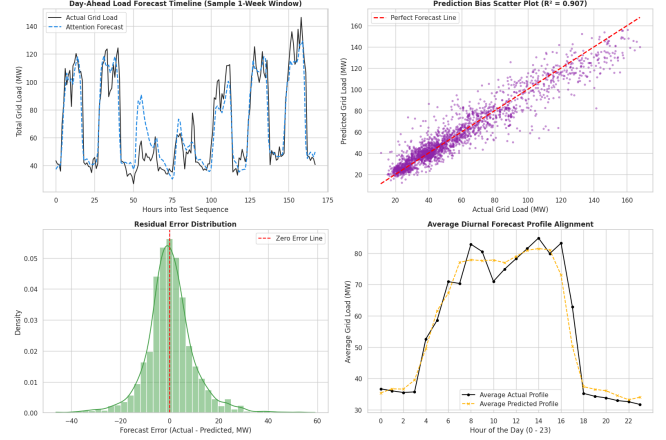


Figure 5: Diagnostic dashboard for the attention Seq2Seq LSTM on the test set.

VI. PROPOSED MODEL: SEQ2SEQ LSTM WITH ATTENTION

Our proposed model is a Seq2Seq LSTM. The encoder uses 64 LSTM units to process 168 hours of historical load, and its final hidden and cell states initialize a 64-unit decoder that generates forecasts for the next 24 hours, one step at a time. We also tried adding a Luong dot-product attention layer, which computes alignment scores between the decoder's current hidden state and all encoder hidden states; the resulting context vector is concatenated with the decoder output and passed through a time-distributed dense layer.

A. Univariate models

First we evaluated the models using only historical load data.

The attention model reached a training loss of 0.1100 and a validation loss of 0.0915, with a step-ahead test MAE of 6.56 MW and a full-horizon R^2 of 0.9286.

The standard model, without attention, had a training loss of 0.1062 and validation loss of 0.1025, with a step-ahead MAE of 6.61 MW and full-horizon R^2 of 0.9244. Its loss ratio of 0.96 was also stable, but the attention variant still edged it out on full-horizon R^2 .

B. Multivariate models

Next we added calendar and weather variables to the encoder input.

The five-feature model combined load with cyclical hour and day-of-week variables. With attention: training loss 0.0670, validation loss 0.1144, MAE 10.01 MW, R^2 0.7938, loss ratio 1.70. Without attention: training loss 0.0721, validation loss 0.1540, MAE 10.26 MW, R^2 0.7928, loss ratio 2.13.

C. Key findings

A few patterns stood out from these experiments.

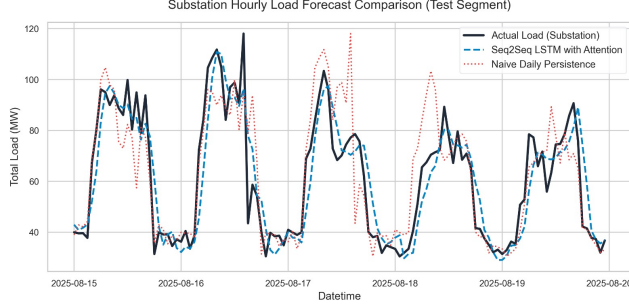


Figure 6: Attention Seq2Seq LSTM forecasts compared to the actual load and a daily persistence baseline over five days in August 2025.

Feeding calendar and weather features directly into the encoder made overfitting worse compared to the univariate models. The five-feature attention model’s validation-to-training loss ratio jumped to 1.70 from 0.83 in the univariate case, and its test R^2 dropped from 0.9286 to 0.7938. The same thing happened without attention: loss ratio up to 2.13, R^2 down to 0.7928. The added features seem to have increased input dimensionality faster than they helped the encoder generalize.

Among the deep learning models, the univariate Seq2Seq LSTM with attention gave the best multi-horizon generalization: lowest error, least overfitting of anything we tested.

VII. COMPARATIVE RESULTS

Table 4 ranks the evaluated models by test R^2 , along with training loss, validation loss, and step-ahead error where available.

Table 4: Performance Summary Across Modeling Paradigms

Model	Configuration	Exogenous Features	Test MAE	Test R^2
Seq2Seq LSTM	Univariate, Luong Attention	None	6.56 MW	0.9286
Seq2Seq LSTM	Univariate, Standard Enc-Dec	None	6.61 MW	0.9244
Seq2Seq LSTM	Multivariate (5 Feat, Attention)	Weather + Lags + Cal.	10.01 MW	0.7938
XGBoost (V3)	max_depth=8, no lag_12	Weather + Cyclical	8.53 MW	0.8517
XGBoost (V3)	max_depth=8, no lag_12	None	8.32 MW	0.8513
SVR	RBF Kernel ($C = 50$)	None	9.44 MW	0.8341
SVR	RBF Kernel ($C = 50$)	Weather	10.24 MW	0.7801

VIII. CONCLUSION

This study evaluated short-term load forecasting models, SARIMA, XGBoost, SVR, stacked LSTM, and Seq2Seq LSTM with and without attention, for a 220kV substation.

The univariate Seq2Seq LSTM with Luong attention generalized best across the 24-hour horizon, resisting overfitting and reaching the highest accuracy (MAE = 6.56 MW, $R^2 = 0.9286$). XGBoost was the strongest baseline and made reasonable use of weather data ($R^2 = 0.8517$), while SVR got worse once exogenous features were added. The stacked recursive LSTM had a low MAE but its errors were autocorrelated.

Adding multivariate features like calendar variables directly to the Seq2Seq encoder caused real overfitting, dropping test R^2 from 0.9286 to 0.7938. Because of that, a reasonable direction for future work is a future-guided architecture, where the encoder sees only historical load and the decoder receives future variables step by step.

In practice, the forecasting horizon is what should drive model choice: tree-based models like XGBoost are cheap and effective for single-step forecasts, while attention-based sequence models generalize better across multiple steps, at the cost of more expensive training.

IX. ACKNOWLEDGEMENTS

We thank Assistant Professor Swathi Battula at IIT Kanpur for her support and for allowing us to undertake this project. We also thank our mentor Naveen Kumar Thokala for sharing resources and guiding us through the technical implementation.

REFERENCES

- [1] DeepLearningAI. Deep Learning Specialization, YouTube Playlist. Available: [YouTube Link](#)
- [2] Indian Energy Exchange. Day Ahead Market Snapshot. Available: [iexindia.com](#)
- [3] P. J. Brockwell and R. A. Davis. Introduction to Time Series and Forecasting. Springer, 2nd Edition.
- [4] C. Olah. “Understanding LSTM Networks,” colah’s blog, Aug. 2015. Available: [colah.github.io](#)
- [5] Greg Hogg. “LSTM Time Series Forecasting Tutorial in Python,” YouTube, 2023. Available: [YouTube Video](#)